

Практическое занятие 20: Создание и отображение простых представлений

Цель занятия:

Научиться создавать различные типы представлений в Django (функциональные и классовые) и отображать контент на страницах сайта.

Задачи занятия

1. Введение в представления Django
 2. Создание функциональных представлений
 3. Создание шаблонов для новых страниц
 4. Создание классового представления
 5. Обновление навигации на сайте
 6. Запуск и проверка приложения
 7. Выполнение практического упражнения. Создание собственного представления
 8. Выполнение дополнительных заданий. Представление списка пользователей, контекстные данные
-

Задача 1: Введение в представления Django

1.1. Что такое представления в Django?

- **Представления (Views)** — это функции или классы, которые принимают HTTP-запрос и возвращают HTTP-ответ.
- Они содержат логику, определяющую, какие данные будут отображаться пользователю и какой шаблон будет использоваться для рендеринга страницы.

1.2. Типы представлений

- **Функциональные представления (Function-Based Views, FBV):**
 - Представляют собой обычные функции Python.
 - Просты и удобны для небольших проектов или простых задач.
- **Классовые представления (Class-Based Views, CBV):**

- Основываются на классах и предоставляют больше возможностей через наследование и миксины.
 - Упрощают повторное использование кода и масштабирование приложения.
-

Задача 2: Создание функциональных представлений

2.1. Открытие файла `events/views.py`

- В VSCode откройте файл `events/views.py`.

2.2. Добавление функциональных представлений `services` и `team`

Добавьте следующие функции в `views.py`:

```
# events/views.py

from django.shortcuts import render

def services(request):
    return render(request, 'events/services.html')

def team(request):
    return render(request, 'events/team.html')
```

Пояснение:

- `services`: Функция для отображения страницы "Услуги".
- `team`: Функция для отображения страницы "Команда".

2.3. Сохраните изменения

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS) для сохранения файла.
-

Задача 3: Добавление маршрутов для новых представлений

3.1. Открытие файла `events/urls.py`

- В папке `events/` откройте файл `urls.py`.

3.2. Добавление маршрутов

Добавьте следующие маршруты в `urlpatterns`:

```
# events/urls.py

from django.urls import path
from . import views

app_name = 'events'

urlpatterns = [
    path('', views.home, name='home'),
    path('events/', views.event_list, name='event_list'),
    path('events/<int:event_id>/', views.event_detail, name='event_detail'),
    path('services/', views.services, name='services'),
    path('team/', views.team, name='team'),
]
```

Пояснение:

- `'services/'`: Маршрут для страницы "Услуги".
- `'team/'`: Маршрут для страницы "Команда".

3.3. Сохраните изменения

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS) для сохранения файла.
-

Задача 4: Создание шаблонов для новых страниц

4.1. Создание директории для шаблонов (если она еще не создана)

- Убедитесь, что в папке `templates/events/` есть место для хранения шаблонов.
- Если папка отсутствует, создайте ее: `templates/events/`.

4.2. Создание шаблона `services.html`

1. Создайте файл `services.html` в `templates/events/`.
2. Добавьте следующий код:

```
<!-- templates/events/services.html -->

<!DOCTYPE html>
```

```

<html>
<head>
  <title>Услуги</title>
  {% load static %}
  <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
  <h1>Наши услуги</h1>
  <p>Описание предоставляемых услуг.</p>
</body>
</html>

```

4.3. Создание шаблона team.html

1. Создайте файл `team.html` в `templates/events/`.
2. Добавьте следующий код:

```

<!-- templates/events/team.html -->

<!DOCTYPE html>
<html>
<head>
  <title>Команда</title>
  {% load static %}
  <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
  <h1>Наша команда</h1>
  <p>Информация о членах команды.</p>
</body>
</html>

```

4.4. Сохраните изменения

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS) для сохранения файлов.

Задача 5: Создание классового представления

5.1. Добавление классового представления `GalleryView`

Откройте файл `events/views.py` и добавьте следующее:

```
# events/views.py

from django.views.generic import TemplateView

class GalleryView(TemplateView):
    template_name = 'events/gallery.html'
```

Пояснение:

- `GalleryView`: Классовое представление для отображения страницы "Галерея".
- Наследуется от `TemplateView` и использует шаблон `gallery.html`.

5.2. Добавление маршрута для `GalleryView`

Откройте файл `events/urls.py` и добавьте маршрут:

```
# events/urls.py

from django.urls import path
from . import views

app_name = 'events'

urlpatterns = [
    path('', views.home, name='home'),
    path('events/', views.event_list, name='event_list'),
    path('events/<int:event_id>/', views.event_detail, name='event_detail'),
    path('services/', views.services, name='services'),
    path('team/', views.team, name='team'),
    path('gallery/', views.GalleryView.as_view(), name='gallery'),
]
```

Пояснение:

- `'gallery/'`: Маршрут для страницы "Галерея", использующий классовое представление `GalleryView`.

5.3. Создание шаблона `gallery.html`

1. Создайте файл `gallery.html` в `templates/events/`.
2. Добавьте следующий код:

```
<!-- templates/events/gallery.html -->

<!DOCTYPE html>
<html>
```

```
<head>
  <title>Галерея</title>
  {% load static %}
  <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
  <h1>Галерея мероприятий</h1>
  <p>Фотографии и видео с мероприятий.</p>
</body>
</html>
```

5.4. Сохраните изменения

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS) для сохранения файлов.

Задача 6: Обновление навигации на сайте

6.1. Добавление навигационного меню

Добавьте следующий код в секцию `<body>` вашего домашнего шаблона (home):

```
<nav>
  <a href="{% url 'events:home' %}">Главная</a> |
  <a href="{% url 'events:services' %}">Услуги</a> |
  <a href="{% url 'events:team' %}">Команда</a> |
  <a href="{% url 'events:gallery' %}">Галерея</a> |
</nav>
```

Дополните код для разделов `about` и `contact`, если они у вас реализованы

Пояснение:

- Навигационное меню с ссылками на основные страницы сайта.
- Используется тег `{% url %}` для создания ссылок на маршруты по их именам.

6.2. Сохраните изменения

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS) для сохранения файлов.

Задача 7: Запуск и проверка приложения

7.1. Запуск сервера разработки

1. Активируйте виртуальное окружение (если не активировано):

- На Windows:

```
venv\Scripts\activate
```

- На macOS/Linux:

```
source venv/bin/activate
```

2. Запустите сервер:

```
python manage.py runserver
```

- **Примечание:** Если команда `python` не работает на macOS/Linux, используйте `python3`.

7.2. Проверка работы новых представлений

1. Перейдите по адресу <http://127.0.0.1:8000/>:

- Убедитесь, что на главной странице отображается навигационное меню.

2. Проверьте страницу "Услуги":

- Перейдите по адресу <http://127.0.0.1:8000/services/>
- Ожидаемый результат: Страница с заголовком "Наши услуги" и описанием.

3. Проверьте страницу "Команда":

- Перейдите по адресу <http://127.0.0.1:8000/team/>
- Ожидаемый результат: Страница с заголовком "Наша команда" и информацией.

4. Проверьте страницу "Галерея":

- Перейдите по адресу <http://127.0.0.1:8000/gallery/>
- Ожидаемый результат: Страница с заголовком "Галерея мероприятий" и контентом.

7.3. Устранение возможных ошибок

- **404 Ошибка:** Проверьте правильность маршрутов и имен представлений.
 - **TemplateDoesNotExist:** Убедитесь, что шаблоны находятся в правильной директории (`templates/events/`) и что пути указаны верно.
-

Задача 8: Практическое упражнение: Создание собственного представления

8.1. Задание для студентов:

- Создайте новое функциональное представление `contact_us` в `events/views.py`.
- Создайте соответствующий маршрут `/contact_us/` в `events/urls.py`.
- Создайте шаблон `contact_us.html` в `templates/events/` с простой формой или текстом.
- Добавьте ссылку на новую страницу в навигационное меню.

8.2. Пример решения:

1. Добавьте функцию `contact_us` в `events/views.py`:

```
# events/views.py

def contact_us(request):
    return render(request, 'events/contact_us.html')
```

2. Добавьте маршрут в `events/urls.py`:

```
# events/urls.py

urlpatterns = [
    # ... другие маршруты ...
    path('contact_us/', views.contact_us, name='contact_us'),
]
```

3. Создайте шаблон `contact_us.html`:

```
<!-- templates/events/contact_us.html -->

<!DOCTYPE html>
<html>
<head>
    <title>Свяжитесь с нами</title>
    {% load static %}
    <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
    <h1>Свяжитесь с нами</h1>
    <p>Здесь вы можете разместить форму обратной связи или контактную
информацию.</p>
```



```
</body>
</html>
```

4. Добавьте ссылку в навигационное меню:

```
<nav>
    <!-- ... другие ссылки ... -->
    <a href="{% url 'events:contact_us' %}">Свяжитесь с нами</a>
</nav>
```

8.3. Проверка работы:

- Перейдите по адресу http://127.0.0.1:8000/contact_us/ и убедитесь, что страница отображается корректно.

Дополнительное задание

1. Создание представления списка пользователей

Цель: Реализовать классическое представление `UserListView`, отображающее список зарегистрированных пользователей.

Шаги:

1. Добавьте представление в `events/views.py`:

```
# events/views.py

from django.views.generic import ListView
from django.contrib.auth.models import User

class UserListView(ListView):
    model = User
    template_name = 'events/user_list.html'
    context_object_name = 'users'
```

2. Добавьте маршрут в `events/urls.py`:

```
# events/urls.py

urlpatterns = [
    # ... другие маршруты ...
    path('users/', views.UserListView.as_view(), name='user_list'),
]
```

3. Создайте шаблон `user_list.html`:

```
<!-- templates/events/user_list.html -->

<!DOCTYPE html>
<html>
<head>
    <title>Список пользователей</title>
    {% load static %}
    <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
    <h1>Список зарегистрированных пользователей</h1>
    <ul>
        {% for user in users %}
            <li>{{ user.username }} ({{ user.email }})</li>
        {% empty %}
            <li>Нет зарегистрированных пользователей.</li>
        {% endfor %}
    </ul>
</body>
</html>
```

4. Добавьте ссылку в навигационное меню:

```
<nav>
    <!-- ... другие ссылки ... -->
    <a href="{% url 'events:user_list' %}">Пользователи</a>
</nav>
```

5. Проверка работы:

- Перейдите по адресу <http://127.0.0.1:8000/users/> и убедитесь, что список пользователей отображается.

2. Добавление контекстных данных

Цель: Передать дополнительные данные в представления `services` и `team`, такие как список услуг или информация о команде.

Поскольку мы еще не используем модели и базы данных, будем использовать статические данные.

1. Обновите функцию `services` в `events/views.py`:

```
# events/views.py

def services(request):
    services_list = [
        'Организация мероприятий',
        'Аренда оборудования',
        'Кейтеринг',
        'Развлекательные программы',
    ]
    return render(request, 'events/services.html', {'services':
services_list})
```

2. Обновите шаблон services.html:

```
<!-- templates/events/services.html -->

<!DOCTYPE html>
<html>
<head>
    <title>Услуги</title>
    {% load static %}
    <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
    <h1>Наши услуги</h1>
    <ul>
        {% for service in services %}
            <li>{{ service }}</li>
        {% endfor %}
    </ul>
</body>
</html>
```

3. Обновите функцию team в events/views.py:

```
# events/views.py

def team(request):
    team_members = [
        {'name': 'Иван Иванов', 'position': 'Директор'},
        {'name': 'Петр Петров', 'position': 'Менеджер проектов'},
        {'name': 'Светлана Смирнова', 'position': 'Координатор
мероприятий'},
    ]
    return render(request, 'events/team.html', {'team': team_members})
```

4. Обновите шаблон `team.html`:

```
<!-- templates/events/team.html -->

<!DOCTYPE html>
<html>
<head>
    <title>Команда</title>
    {% load static %}
    <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}" %">
</head>
<body>
    <h1>Наша команда</h1>
    <ul>
        {% for member in team %}
            <li>{{ member.name }} - {{ member.position }}</li>
        {% endfor %}
    </ul>
</body>
</html>
```

5. Проверка работы:

- Перейдите на страницы "Услуги" и "Команда" и убедитесь, что данные отображаются корректно.

Заключение

Поздравляем! Вы успешно:

- Создали функциональные и классовые представления в Django.
- Настроили маршруты для новых представлений.
- Создали шаблоны и отобразили контент на страницах сайта.
- Обновили навигационное меню для доступа к новым страницам.
- Передали и отобразили контекстные данные в шаблонах.

Если у вас возникнут дополнительные вопросы или потребуется помощь, не стесняйтесь обращаться к преподавателю или обсуждать с одногруппниками.

Успехов в разработке!

Вопросы для обсуждения

- **Чем отличаются функциональные и классовые представления?**
 - Функциональные представления проще и легче для понимания, но классовые предоставляют больше возможностей для расширения и повторного использования кода.
 - **Когда стоит использовать классовые представления?**
 - Когда вам нужно реализовать стандартные операции (например, отображение списка объектов), или когда вы хотите воспользоваться наследованием и миксинами для добавления функциональности.
 - **Как передавать данные из представления в шаблон?**
 - Через контекст — словарь, который передается в функцию `render()` и доступен в шаблоне.
-

Полезные ресурсы

- [Документация Django: Представления](#)
- [Классовые представления Django](#)
- [Django Template Language](#)
- [Документация VSCode](#)